后训练学习计划 • 第四周（9/28 - 10/4）

本周定位：**Phase 1 第三周——把「会写模型」变成「会跑训练」**。主题是 PyTorch 的训练系统：数据管线、训练循环、checkpoint 续训、手写 AdamW 对拍、lr 调度、混合精度、梯度累积与裁剪、显存解剖。前三周你在算 loss、在造产生 loss 的结构，这周你要把两者接上电，并且**看得懂电流过的每一行**。

用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，`> 注` 块 = 注释（可跳过，不影响主线）。本周闭合 Week 3 明确推过来的四个洞（`block.py` 与 nanoGPT 未对齐、`model.train()/eval()`、AMP/累积/checkpoint/调度器、Pre-LN 稳定性只讲不练）；不发博客（配额在 Week 5），但「显存/速度对照表」和「踩坑记录」是 Week 5 博客与 Week 14 失败案例清单的直接原料。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	`get_batch` 手写 → 自建 `Dataset` / `DataLoader`	`data_lab.py`：两种取批都能打印 `(B,T)` 且 `y` 恰好错位一位	6h
Day 2 周二	训练循环 v1：train/eval、`no_grad`、checkpoint、断点续训	`RESUME OK`（恢复 rng 后逐位相同，不恢复明显跑偏）	6.5h
Day 3 周三	手写 AdamW 对拍、优化器显存代价、warmup+cosine 调度	`ADAMW CROSSCHECK OK`（3 个 lr，偏离 <1e-6）+ `lr_sched.png`	6h
Day 4 周四	bf16 vs fp16+GradScaler、梯度累积、梯度裁剪	`ACCUM EQUIV OK / CLIP SEMANTICS OK` + 精度对照小表	6.5h
Day 5 周五	显存解剖表、gradient checkpointing、DDP/ZeRO 概念	手算 vs 实测显存表（误差有解释）+ 速度/显存对照表	6h
Day 6 周六	`train_template.py` 定稿真跑、pre/post 稳定性对照、自测、push	4090 上 1000 步跑通 + 续训复现 + GitHub 链接	6.5h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + Week 5（M1）预习	验收 ≥6/7 项通过	4h

注：**本周卡时上限 8h、计划用 6h、预估花费 ¥9-15**（AutoDL 4090 约 ¥1.5-2.5/h；A100/H800 是 ¥6-12/h，本周的活不需要它，别顺手开贵卡）。用途分配：Day 5 约 2.5h（显存实测 + checkpointing 对照），Day 6 约 2.5h（1000 步真跑 + pre/post 对照 + 续训复现），**留 1.5h 余量**给重跑。超了就砍 Day 6 任务 2 的第二组，别砍 Day 5 的实测——**手算与实测对不上号的那一晚，是这周最长进的一晚**。P1 卡时上限 30h：Week 2 用了 0h、Week 3 用 ≤2h、本周 ≤8h、Week 5 预留 12h。

注：本周最大风险是**「跑通了」冒充「懂了」**。判据里我全部要求可打印、可对拍的东西，就是为了堵这条：不写 checkpoint 一致性测试，你会以为续训是对的，而它悄悄改了学习率流；不做累积等价性测试，你会带着一个 4 倍 lr 的错误进 Week 5 的大训练。第二风险是**Windows 下 DataLoader 多进程**——`num_workers>0` 必须有 `if __name__ == "__main__":` 保护，否则进程反复重开、卡死、没有任何报错。

注：本周**故意不含**：真实模型训练（Week 5 M1）、LoRA/SFT（Week 6）、DDP/ZeRO 实操（总计划的分布式补强档，若转社招再加）、FSDP/DeepSpeed 一个都不碰。概念只到「能讲清切分对象」。

二、开始前：环境、卡、上周欠账（约 30 分钟，前一晚做完）

1. 环境仍是 `E:/study/posttrain/.venv`（**别在 conda base 里跑 torch**：Windows 下 conda 的 MKL 与 torch 自带 OpenMP 打架，报 `OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized`；临时绕过 `set KMP_DUPLICATE_LIB_OK=TRUE`，正解是回 venv——这条我今天在你身上验证过两次，装完 matplotlib 再 import torch 一样会炸）：

```
cd E:/study/posttrain
source .venv/Scripts/activate          # PowerShell 用 .venv\Scripts\Activate.ps1
pip install "numpy>=1.24" matplotlib tqdm -i https://pypi.tuna.tsinghua.edu.cn/simple
python -c "import torch,numpy,matplotlib;print(torch.__version__, numpy.__version__)"
mkdir -p E:/study/posttrain/week04/{day1,day2,day3,day4,day5,day6,ckpt}
ls E:/study/posttrain/week03/block.py                        # 本周要把它塞进训练循环
ls E:/study/posttrain/nanoGPT/data/shakespeare_char/         # Week 3 跑过 prepare.py,train.bin/val.bin 在这
```

2. **周四晚之前把卡订好**（Day 5/Day 6 用）：AutoDL 选 4090、按量计费、镜像选「PyTorch 2.x / CUDA 12.x」基础镜像。**数据与 checkpoint 一律放** `/root/autodl-tmp`（系统盘会被镜像重置清掉）；**关机 ≠ 销毁实例**，但释放实例会丢系统盘，所以 `/root/autodl-tmp` 之外的东西都要有下载回来的手段。
3. 打开 `E:/study/posttrain/week03/week03-gaps.md`（Day 1 任务 1 按它认领）。

完成判据：上面 bash 块 6 行全部成功、`python -c` 那行**没有** `OMP: Error #15`、`week04/` 七个子目录存在、`shakespeare_char/` 里能看到 `train.bin` 与 `val.bin`（没有就现在跑一次 `python data/shakespeare_char/prepare.py`）、AutoDL 已订卡或已确认周末有货。

Day 1 • 周一：数据管线——先自己写一遍 `get_batch`，再换 DataLoader，然后说清差别

今日总览（6h）：模型你已经会写了，但训练的第一行代码不是 `model = GPT(cfg)`，是「这一步喂什么」。今天你要：① 手写 nanoGPT 那种 `get_batch`（`np.memmap` + 随机起点）；② 写一个正经的 `Dataset` + `DataLoader`；③ 用同一个 seed 让两者给出可比的批次，并把「为什么 nanoGPT 不用 DataLoader」写成有数据的结论，而不是抄一句「DataLoader 慢」。

任务 1：开工对账——从 `week03-gaps.md` 里清哪几条（约 20 分钟）

打开 `week03/week03-gaps.md`，把条目抄进 `week04/day1/today.md` 并逐条归派：

1. `block.py` **还没和 nanoGPT 完全对齐（初始化 std、dropout、weight tying、`ln_f`）** → 本周 Day 2（把它变成能存 checkpoint 的 `nn.Module`）+ Day 6（真跑时逐项对齐）。
2. **BatchNorm 与** `model.train()/eval()` **的行为差别** → 本周 Day 2 任务 2，用 dropout 做同样的实验（BN 本身不在 decoder 里，讲清机制即可）。
3. **混合精度、梯度累积、checkpoint、lr 调度器** → 本周 Day 2（checkpoint）/ Day 3（AdamW、调度器）/ Day 4（AMP、累积、裁剪）。

4. **Pre-LN vs Post-LN 稳定性只讲不练** → 本周 Day 6 任务 2。
5. **KV cache 我只算了没实现、BPE 真实分词、采样参数对照** → **Week 5**，本周不碰。
6. Week 3 的遗留未勾项（若有）：今天能清就清，不能的写明「挪到 Week N + 哪天」。

完成判据：`today.md` ≥6 行认领，其中第 1-4 条写明本周哪天，第 5 条写明 Week 5；第 6 条若存在必须有「今天做掉」或「挪到 Week N」的结论，不接受「之后补」。

> 注：Week 3 的 gaps 表里那几条「归到 Week 4」的，今天全部有了具体日子。**没有日子的问题等于没有安排**——这句话从这周起对你所有的事生效，包括 Week 13 的投递台账。

任务 2：数据就位 + 重看 `get_batch` 那一段（约 55 分钟）

1. 用 Week 3 已准备好的字符级数据：`nanoGPT/data/shakespeare_char/{train.bin,val.bin,meta.pkl}`。用 `np.memmap(..., dtype=np.uint16, mode='r')` 打开，打印长度；从 `meta.pkl` 里读 `stoi/itos/vocab_size`（**以你打印出来的为准**，字符级一般在 65 上下）。
2. 回看《Let's build GPT》里 `get_batch` 那 15 分钟 + nanoGPT `train.py` 的 `get_batch` 源码（`grep -n "def get_batch" train.py`），回答三件事：它返回的 `yb` 和 `xb` 是什么关系？`np.random.randint` 的上界为什么减 `block_size + 1`？为什么用 `memmap` 而不是一次读进内存？
3. NOTES 记下三个真实数字：`len(train.bin)`、`len(val.bin)`、`vocab_size`。

完成判据：打印出三个数字且 `len(val.bin) < len(train.bin)`；NOTES 里三问都有答案（尤其第二问要写成「防止取到越界的窗口」这类明确说法）；能口头说出「`xb[:, 1:] == yb[:, :-1]`」这条不变式。

任务 3：手写 `get_batch`（约 90 分钟）

新建 `week04/day1/data_lab.py`（**这个文件今天写两次取批，Day 2 的训练循环要 import 它**）：

```
import pickle, numpy as np, torch
DATA_DIR = "E:/study/posttrain/nanoGPT/data/shakespeare_char"
with open(f"{DATA_DIR}/meta.pkl", "rb") as f:
    meta = pickle.load(f)
VOCAB = meta["vocab_size"]
def load(split):
    return np.memmap(f"{DATA_DIR}/{split}.bin", dtype=np.uint16, mode="r")
def get_batch(data, block_size=64, batch_size=32, rng=np.random.default_rng(0)):
    ix = rng.integers(0, len(data) - block_size - 1, size=batch_size)   # 起点上界要留够窗口
    xb = np.stack([data[i:i + block_size] for i in ix]).astype(np.int64)
    yb = np.stack([data[i + 1:i + block_size + 1] for i in ix]).astype(np.int64)
    return torch.from_numpy(xb), torch.from_numpy(yb)
if __name__ == "__main__":
    tr, va = load("train"), load("val")
    xb, yb = get_batch(tr)
    print("shapes", tuple(xb.shape), tuple(yb.shape), "shift ok:", bool((yb[:, :-1] == xb[:, 1:]).all()))
    print("decode:", "".join(meta["itos"][int(i)] for i in xb[0]))
    print("GETBATCH OK", xb.dtype, int(xb.min()), int(xb.max()), VOCAB)
```

完成判据：打印 `GETBATCH OK`、`shift ok: True`、`torch.int64`，且 `0 <= xb.max() < VOCAB`（dtype 不是 int64 时 `nn.Embedding` 会报 `Expected tensor for argument #1 'indices' to have scalar type Long`，这是今天的第一

个坑）；解码出的 200 字符能看出是莎士比亚。

> 注：`get_batch` 是「每步现采」而不是「遍历数据集」——它永远不重复地随机切窗口，所以没有 epoch 概念。这是 nanoGPT 的选择，不是唯一正确的选择：数据集小、要精确复现每一批时你需要 `Dataset`。今天下午两种都写，就是为了让你有资格说「我按情况选」。

任务 4：自建 `Dataset` + `DataLoader`（约 110 分钟）

在同一文件里加 `CharLMDataset`：

```
from torch.utils.data import Dataset, DataLoader
class CharLMDataset(Dataset):
    def __init__(self, split, block_size=64):
        self.data = load(split); self.t = block_size
    def __len__(self):
        return len(self.data) - self.t - 1            # 每个起点一个样本,和 get_batch 的窗口约定一致
    def __getitem__(self, i):
        c = np.asarray(self.data[i:i + self.t + 1], dtype=np.int64)
        return torch.from_numpy(c[:-1]), torch.from_numpy(c[1:])
if __name__ == "__main__":
    ds = CharLMDataset("train")
    dl = DataLoader(ds, batch_size=32, shuffle=True, num_workers=0, drop_last=True)
    xb, yb = next(iter(dl))
    print("DL", tuple(xb.shape), bool((yb[:, :-1] == xb[:, 1:]).all()), len(dl))
    a = next(iter(dl))[0]; b = next(iter(dl))[0]
    print("同一次迭代相邻两批是否相同:", torch.equal(a, b))
    print("DATALOADER OK")
```

三条必须今天搞清的语义（写进 NOTES，每条配一个你打印出来的数）：① `shuffle=True` 时两个**完整 epoch** 的批次顺序不同、但覆盖的样本集合相同；② `drop_last=True` 会丢掉末尾不足 `batch_size` 的那一撮，`len(dl)` 因此等于 `(len(ds) // 32)` 而不是 `ceil`；③ **`num_workers=0` + `if __name__ == "__main__":` 保护**是 Windows 上唯一的省事做法。

完成判据：打印 `DATALOADER OK`、形状 `(32, 64)`、`shift` 断言 `True`、`torch.equal(a, b)` 为 `False`、`len(dl)` 与手算 `len(ds) // 32` 相等（不等就是 `drop_last` 忘了）；把 `num_workers` 改成 2 且不删 `__main__` 保护跑一次，**把卡死/报错的现象记进踩坑记录**（跑不完就 Ctrl+C，别等）。

> 注：`num_workers>0` 时子进程会重新 import 你的主模块，Windows 没有 `fork`、只有 `spawn`，没有 `__main__` 保护就是无限套娃起进程。**这不是 PyTorch 的坑，是 Windows 的进程模型**——面试问到「DataLoader 多进程为什么在 Windows 上容易挂」，能答到 spawn/重导入这一层的候选人不多。

任务 5：两种取批的对照与选型结论（约 55 分钟）

在 `data_lab.py` 里加一段计时：同一个 `block_size=64, batch_size=32`，各取 200 批，打印两个墙钟时间与比值（`time.perf_counter()`，先跑 20 批预热再计时）。再回答：如果 `__getitem__` 里换成「字符串解码 + 正则清洗 + 重新编码」，比值会变成什么？为什么？

完成判据：打印两个毫秒数与比值，NOTES 有一句**带数字的选型结论**（例如「这份数据上两者接近，但一旦样本要走 tokenizer，DataLoader 的多进程才有价值，字符级用 `get_batch` 足够」——你的结论必须由你跑出的比

值支撑，抄我的不算）；再写一句「Week 6 的 SFT 数据属于哪一类」。

> 注：别为了「让 DataLoader 赢」去调 `pin_memory`、`prefetch_factor`。今天的目的是**知道每个参数在干什么**，Week 6 你会因为真实 tokenizer 变慢而自然地用上多进程，那时你已经知道 Windows 的坑在哪。

任务 6：收工笔记（约 30 分钟）

`week04/day1/notes.md`：今天两次报错的原文 + 修法；`data_lab.py` 的 `get_batch` 与 `CharLMDataset` 各自的「窗口起点上界」是什么、为什么必须一致（不一致会导致 Day 2 的续训测试假通过）。

完成判据：NOTES ≥3 条且含 2 条报错原文；写清两个上界表达式（`len(data) - block_size - 1` 与 `len(data) - t - 1`）并说明为什么等价；明天要用的东西（`train/val` 划分、`eval` 迭代次数）已在 NOTES 里列出。

今日验收（6h）：`GETBATCH OK` + `DATALOADER OK` + `len(dl)` 与手算相等 + 一个带数字的选型结论 + gaps 认领 6 行。

Day 2 • 周二：训练循环 v1——train/eval、`no_grad`、checkpoint、断点续训

今日总览（6.5h）：今天要写出一个你以后每个项目都会抄的模板。硬指标只有一个：**同一段代码跑 200 步的 loss 序列，与「跑 100 步存档 → 重新加载 → 再跑 100 步」的 loss 序列逐位相同**。做不到，说明你的 checkpoint 少了东西——而少了东西的续训在 Week 5 会让你以为是自己调错了 lr。

任务 1：手写最小训练循环（约 90 分钟）

新建 `week04/day2/train_v1.py`：从 `week04/day1/data_lab.py` 里 `import get_batch`，模型先用一个 3 层的小字符 LM（`nn.Embedding` + 两个你 Week 3 写的 `Block` + `ln_f` + `lm_head`，`n_embd=64, n_head=4, block_size=64`）。循环要有：`iter_num`、`lossf`（窗口平均）、`t0`、每 20 步 `it/s`、eval 分支（后两个任务再加）。

```
for iter_num in range(max_iters):
    xb, yb = get_batch(train_data, block_size, batch_size, rng)
    # 训练前向不包 no_grad:要留计算图给 backward;只在 eval 分支里才包
    logits = model(xb)
    loss = torch.nn.functional.cross_entropy(logits.view(-1, V), yb.view(-1))
    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()
    if iter_num % 20 == 0:
        print(iter_num, f"{loss.item():.4f}")
```

完成判据：跑 200 步（CPU 上几分钟可接受），loss 从 `ln(V)`（约 4.17）稳定下降，打印的最后一个数 **≤ 起始值的 80%**；`opt.zero_grad(set_to_none=True)` 存在（少了 `zero_grad` loss 会不降反升，这是今天的必踩坑，故意先试一次不加的）；NOTES 记下 200 步的耗时与最终 loss。

> 注：`set_to_none=True` 不只是省显存——它把 `.grad` 置成 `None` 而不是零张量，于是 `if p.grad is not None` 这类判断的语义会变，Week 6 做梯度累积日志时会撞上。先知道，别现在深究。

任务 2：`model.train()` / `model.eval()` 与 `torch.no_grad()` ——把两件事分开（约 85 分钟）

很多人把这两件事混为一谈，今天用实验拆开：模型里放一个 `nn.Dropout(0.3)`（`p=0` 时看不出差），固定一个输入 `x0`：

```
model.eval()
a = model(x0).clone(); b = model(x0).clone()          # 同一模型同一输入两次前向
model.train()
c = model(x0).clone(); d = model(x0).clone()
print("eval 两次是否相同:", torch.allclose(a, b))      # True
print("train 两次是否相同:", torch.allclose(c, d))     # False
with torch.no_grad():
    out = model(x0)                                     # 前向仍然可以是 train 模式
print("no_grad 下 requires_grad:", out.requires_grad)   # False
```

完成判据：四行打印依次 `True / False / False`；NOTES 用两句话分别写清「`eval()` 改变的是**层的计算行为**（dropout/BN 这类有状态层）」和「`no_grad()` 改变的是**是否建图**」，并各配今天的一个打印值当证据；再加一句「训练模式 + `no_grad()` 是可能的组合，但 eval 必须两者一起用」。

> 注：LayerNorm / RMSNorm **在 train/eval 下行为完全相同**（它对 batch 和序列都不敏感），BatchNorm 才会用 running stats 切换——这就是 decoder-only Transformer 选 LN 的根本理由，也是为什么本模板里 `eval()` 只对 dropout 起作用。这一条同时闭合 Week 2 遗留的 BN 洞：机制你懂了，只是这套架构里用不上它。

任务 3：checkpoint 与断点续训一致性——今天的硬仗（约 120 分钟）

checkpoint 至少要存五样：`model.state_dict()`、`optimizer.state_dict()`、`iter_num`、`torch.get_rng_state()`、以及 config（或能重建模型的东西）。nanoGPT 还存 `best_val_loss`；`register_buffer` 的东西（Week 3 那个 causal `bias`）**在 `state_dict` 里但不在 `parameters()` 里**，这正是必须存 state_dict 的理由。

`week04/day2/resume_test.py`（本地 CPU 就能跑，用小模型）：

```
import numpy as np, torch, torch.nn as nn
g = torch.Generator().manual_seed(4242)              # 数据流独立于全局流,三组实验共用同一批数据
XS = torch.randn(64, 16, generator=g); YS = torch.randint(0, 4, (64,), generator=g)
class Tiny(nn.Module):
    def __init__(self):
        super().__init__()
        self.a = nn.Linear(16, 32); self.drop = nn.Dropout(0.2); self.b = nn.Linear(32, 4)
    def forward(self, x):
        return self.b(self.drop(torch.relu(self.a(x))))
def build():
    torch.manual_seed(42)                            # 模拟"新进程从同一种子起步"
    model = Tiny()
    return model, torch.optim.AdamW(model.parameters(), lr=1e-2)
def run(model, opt, n0, n1, log):
    for _ in range(n0, n1):
        loss = nn.functional.cross_entropy(model(XS), YS)
        opt.zero_grad(); loss.backward(); opt.step(); log.append(loss.item())
A = []; m, o = build(); run(m, o, 0, 200, A)
B = []; m, o = build(); run(m, o, 0, 100, B)
ckpt = {"model": m.state_dict(), "opt": o.state_dict(), "iter_num": 100, "rng": torch.get_rng_state()}
m2, o2 = build(); m2.load_state_dict(ckpt["model"]); o2.load_state_dict(ckpt["opt"])
torch.set_rng_state(ckpt["rng"])                     # 少了这一行,dropout 掩码流就接不上
run(m2, o2, 100, 200, B)
C = list(B[:100]); m3, o3 = build(); m3.load_state_dict(ckpt["model"]); o3.load_state_dict(ckpt["opt"])
run(m3, o3, 100, 200, C)                             # 故意不恢复 rng
d_with = np.abs(np.array(A) - np.array(B)).max(); d_wo = np.abs(np.array(A) - np.array(C)).max()
print("恢复 rng 的 max|A-B| =", d_with); print("不恢复 rng 的 max|A-B| =", d_wo)
assert d_with < 1e-6 and d_wo > 1e-4, "两个数没拉开 -> 你的循环里没有随机性,把 dropout 打开再试"
print("RESUME OK")
```

完成判据：打印 `RESUME OK`；「恢复 rng」那一行的 diff **应为 0.0 或 1e-15 量级**（我这边是 0.0，逐位相同）；「不恢复」那一行我这边约 0.12，你的不会一样，但必须 **>1e-4**——两个数拉不开说明循环里根本没有随机性，测试是假的；把「续训必须存的四样」写进 `week04/notes-training.md`。

> 注：**为什么不存 rng 也「看起来正常」**：loss 只是有点抖，没人能凭眼看出来。真正出事的是 Week 5——你续训后 loss 比连续训练高 0.03，第一反应会是「lr 调度接错了」，白排查半天。今天这个「故意做坏一组」的对照，就是为了让这种故障以后 30 秒内能被识别。

任务 4：随机种子全套（约 50 分钟）

写一个 `set_seed(seed)`：`random.seed` / `np.random.seed` / `torch.manual_seed` / `torch.cuda.manual_seed_all`，并在 `train_v1.py` 顶部调用。然后做两个实验：① 同一 seed 连跑两次，前 20 步 loss 是否逐位相同；② 加 `torch.use_deterministic_algorithms(True)`（`CUBLAS_WORKSPACE_CONFIG` 不设值时 CUDA 上会直接报错）看是否变慢、报错文案是什么。

完成判据：实验 ① 打印 `True`（不等就找出是哪一处随机性没被 seed 管住——通常是 `get_batch` 里自带的 `rng`）；实验 ② 的报错原文或「CPU 上无变化」的结论记进 NOTES；NOTES 有一句「为什么『完全可复现』在生产里常常不值得（确定性 kernel 更慢）」，并给出你测到的耗时差（CPU 上可能是 0，如实写 0）。

任务 5：把 checkpoint 接进 `train_v1.py` + 收工（约 45 分钟）

给 `train_v1.py` 加：每 `save_interval` 步存 `week04/ckpt/step{iter_num}.pt`（含任务 3 的五样），`--resume <path>` 参数（`argparse`）恢复后**从** `iter_num` **继续**并把 lr 调度（明天加）对齐。

完成判据：本地跑 `python train_v1.py --max_iters 60`，再 `python train_v1.py --resume week04/ckpt/step30.pt --max_iters 60`，两次打印的第 31-60 步 loss **逐位相同**（把两组数贴进 NOTES 对比，不许只写「一致」）；`--resume` 后打印的起始 `iter_num` 是 30 而不是 0。

今日验收（6.5h）：loss 从 `ln(V)` 降到 ≤80% + `True/False/False` 三行符合 + `RESUME OK`（两个数拉开）+ 续训 31-60 步逐位相同。

Day 3 • 周三：手写 AdamW 与 `torch.optim.AdamW` 逐迭代对拍 + 优化器显存 + lr 调度

今日总览（6h）：Week 2 你推过 `p-y` 和偏差修正的数学，今天验证「你真的写出来了」：同一个初始参数、同一批数据，你的 AdamW 和 PyTorch 的 AdamW **每一步的 loss 与最终参数都要对上**（float64 下应在 1e-15 量级，门槛放 1e-6）。然后把优化器状态的显存代价算成一个数，最后手写 warmup+cosine 与 `LambdaLR` 对拍。

任务 1：手写 AdamW 对拍（约 130 分钟）

`week04/day3/adamw_check.py`（**float64、seed 写死、3 个 lr**）：

```
import torch
torch.manual_seed(3)
X = torch.randn(128, 8, dtype=torch.float64)
y = (X[:, 0] * 2 - X[:, 1] + X[:, 2] ** 2 * 0.5 > 0).to(torch.float64).unsqueeze(1)
class Net(torch.nn.Module):
    def __init__(self):
        super().__init__()
        self.a = torch.nn.Linear(8, 16, dtype=torch.float64)
        self.b = torch.nn.Linear(16, 1, dtype=torch.float64)
    def forward(self, z):
        return torch.sigmoid(self.b(torch.tanh(self.a(z))))
def loss_of(m):
    return torch.nn.functional.binary_cross_entropy(m(X), y)
def mine(step, lr, b1=0.9, b2=0.999, eps=1e-8, wd=1e-2):
    torch.manual_seed(1234)                    # 两边必须同一份初始化，否则对拍没有意义
    model = Net(); m = [torch.zeros_like(p) for p in model.parameters()]
    v = [torch.zeros_like(p) for p in model.parameters()]
    for t in range(1, step + 1):                        # t 从 1 开始,偏差修正才是 /(1-b^t)
        for p in model.parameters(): p.grad = None
        loss_of(model).backward()
        with torch.no_grad():
            for i, p in enumerate(model.parameters()):
                g = p.grad
                m[i].mul_(b1).add_(g, alpha=1 - b1)
                v[i].mul_(b2).addcmul_(g, g, value=1 - b2)
                mh = m[i] / (1 - b1 ** t); vh = v[i] / (1 - b2 ** t)
                p.mul_(1 - lr * wd)                     # AdamW:衰减解耦,乘在参数上而不是塞进梯度
                p.addcdiv_(mh, vh.sqrt() + eps, value=-lr)
    return model
def theirs(step, lr, b1=0.9, b2=0.999, eps=1e-8, wd=1e-2):
    torch.manual_seed(1234)
    model = Net(); opt = torch.optim.AdamW(model.parameters(), lr=lr, betas=(b1, b2), eps=eps,
weight_decay=wd)
    for _ in range(step):
        opt.zero_grad(); loss_of(model).backward(); opt.step()
    return model
if __name__ == "__main__":
    for lr in (1e-3, 1e-2, 3e-2):
        ma = mine(60, lr); tb = theirs(60, lr)
        d = max((p - q).abs().max().item() for p, q in zip(ma.parameters(), tb.parameters()))
        print(f"lr={lr:g}: 最终参数最大偏离 = {d:.3e}")
        assert d < 1e-6, "对不上:先查 t 是否从 1 开始、wd 是否解耦、eps 在根号内还是根号外"
    print("ADAMW CROSSCHECK OK")
```

完成判据：打印 `ADAMW CROSSCHECK OK`，三个 lr 的偏离都 **<1e-6**（float64 下我这边是 1e-16 ~ 3e-15 量级，你落在附近都算过）；把三个数原样抄进 NOTES；再手动把 `p.mul_(1 - lr * wd)` 挪成 `g.add_(p, alpha=wd)`（即 L2 正则 = Adam 而非 AdamW）跑一次，打印偏离量级并解释为什么此时 torch 版和你的版不再是同一件事。

> 注：`torch.optim.AdamW` 的默认 `weight_decay` 是 **0.01**（不是 0），`betas=(0.9, 0.999)`、`eps=1e-8`，与上面完全对齐才有得对拍。这类「默认值不一致导致的静默不等价」是 Week 6 之后调库最常浪费时间的来源，养成**先看默认参数再对拍**的手感。

任务 2：偏差修正复现 + 优化器状态的显存代价（约 55 分钟）

① 用 `m=0.1, v=0.001, b1=0.9, b2=0.999, t=1` 手算 $\hat{m}$、$\hat{v}$，再和 `g` 比：偏差修正的意义就是让「初始为 0 的滑动平均」第一步的估计仍是无偏的。② 算一个数：**AdamW 每个参数额外占多少字节**（`m` + `v`，fp32 各 4B），乘上 nanoGPT 124M 参数量，写出 GB。

完成判据：手算 $\hat{m} = \hat{v} = 1.0$（因为 `1-0.9=0.1`、`1-0.999=0.001`，两个数正好被除掉）并打印验证；NOTES 写明「AdamW 优化器态 = 8 B/参数，124M 模型 ≈ 1.0 GB」，且这句话在 Day 5 的显存解剖表里要被引用一次。

> 注：第一步更新量其实约等于 `lr`（$\hat{m}/\sqrt{\hat{v}} \approx g/|g|$），所以早期更新幅度对梯度大小不敏感、**接近符号更新**。这正是「warmup 前几步 lr 不敢开太大」的机制解释，也是 Week 5 故意用大 lr 制造发散时你会看到的现象。

任务 3：手写 warmup + cosine 调度器并与 `LambdaLR` 对拍（约 105 分钟）

`week04/day3/sched_check.py`（本地 CPU，float64 无必要，纯标量比较）：

```
import math, torch
warmup, total, min_frac, lr0 = 200, 2000, 0.1, 1e-3
def manual_lr(t):                                   # t 从 0 数:第 t 步该用的 lr
    if t < warmup:
        return lr0 * (t + 1) / warmup
    p = (t - warmup) / (total - warmup)
    return lr0 * (min_frac + (1 - min_frac) * 0.5 * (1 + math.cos(math.pi * p)))
def lam(t):                                          # 交给 LambdaLR 的乘子
    if t < warmup:
        return (t + 1) / warmup
    p = (t - warmup) / (total - warmup)
    return min_frac + (1 - min_frac) * 0.5 * (1 + math.cos(math.pi * p))
opt = torch.optim.SGD(torch.nn.Linear(2, 2).parameters(), lr=lr0)
sch = torch.optim.lr_scheduler.LambdaLR(opt, lam, last_epoch=-1)   # 构造时已经推进一步
mine, ref = [], []
for t in range(total):
    ref.append(opt.param_groups[0]["lr"]); mine.append(manual_lr(t)); sch.step()
d = max(abs(a - b) for a, b in zip(mine, ref))
print("max|manual - LambdaLR| =", d)
assert d < 1e-12, "少写 last_epoch=-1 会让整条曲线平移一步"
print("lr[0]=%.6f lr[warmup-1]=%.6f lr[total-1]=%.6f" % (mine[0], mine[warmup - 1], mine[-1]))
import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
plt.plot(mine, label="manual"); plt.plot(ref, ls="--", label="LambdaLR")
plt.axvline(warmup, color="gray", lw=0.6); plt.legend(); plt.xlabel("iter"); plt.ylabel("lr")
plt.savefig("E:/study/posttrain/week04/day3/lr_sched.png", dpi=90); print("SCHED CROSSCHECK OK")
```

完成判据：打印 `SCHED CROSSCHECK OK`、diff <1e-12；三个端点数与手算一致（`lr[0]=5e-6`、`lr[warmup-1]=1e-3`、`lr[total-1]=1e-4`）；`lr_sched.png` 存在且肉眼能看到「直线上升 200 步 + 余弦下降到 `0.1×lr0`」；NOTES 写清 `last_epoch=-1` 那个坑与 `min_frac` 存在的理由（不让 lr 归零，后期仍在学）。

任务 4：接进 `train_v1.py` + 收工（约 70 分钟）

把任务 3 的 `manual_lr` 接成「每步显式赋值 `opt.param_groups[0]['lr']`」的写法（nanoGPT 就是这么干的，避免和续训打架），并做一致性检查：`--max_iters 400` 连续跑的 lr 序列，与「跑 200 存档 → 续跑 200」的序列**逐位相同**（lr 必须由 `iter_num` 决定，不能由「第几次调用 scheduler」决定）。

完成判据：两次打印的 lr 序列前 400 项逐位相同（贴 5 行到 NOTES 作证据）；NOTES 有一句「为什么我用 `iter_num` 驱动 lr 而不是 `scheduler.step()`」；`week04/day4` 待办已列出（bf16/累积/裁剪）。

> 注：`LambdaLR` 不是不能用，而是它的状态挂在调度器对象里，**你必须在 checkpoint 里额外存 `sch.state_dict()`** 才能续训对齐。多一个必须记住的存档项，就多一个出错点——显式按 `iter_num` 算 lr 更笨但更稳，这类「选笨办法并说清理由」的判断，是面试里区分「跑过实验」和「只调过库」的地方。

今日验收（6h）：`ADAMW CROSSCHECK OK`（3 个 lr <1e-6）+ `m̂=v̂=1.0` 手算 + AdamW 状态 8B/参数 + `SCHED CROSSCHECK OK` + 续训 lr 序列逐位相同。

Day 4 • 周四：混合精度、梯度累积、梯度裁剪——三件让大模型跑得动的事

今日总览（6.5h）：今天全部本地 CPU 可做（数值等价性不需要 GPU，卡只需要在晚上确认一遍 bf16 的速度差）。三件事各自有一条硬判据：① fp16/bf16 的**范围与精度**差别用你手上的小张量跑出来；② 累积 4 步 ≡ 整批一步（偏离 <1e-9），并且亲手制造一次「忘了除 accum」在 SGD 和 AdamW 下的不同后果；③ `clip_grad_norm_` 返回的是裁剪**前**的 global norm，用第二个阈值亲眼看到它被压到 0.1。

任务 1：fp16 与 bf16 的数值解剖（约 120 分钟）

`week04/day4/dtype_lab.py`（本地 CPU）：

```
import torch
torch.manual_seed(0)
g = torch.randn(4096) * 3e-5                          # 典型深层小梯度
print("fp16 归零比例", (g.half().float() == 0).float().mean().item())
print("bf16 归零比例", (g.bfloat16().float() == 0).float().mean().item())
print("fp16 相对误差中位数", ((g.half().float() - g).abs() / g.abs()).median().item())
print("bf16 相对误差中位数", ((g.bfloat16().float() - g).abs() / g.abs()).median().item())
print("70000 -> fp16:", torch.tensor(70000.0).half().item(), "| bf16:",
torch.tensor(70000.0).bfloat16().item())
print("1e-5   -> fp16:", torch.tensor(1e-5).half().item(), "| bf16:", torch.tensor(1e-
5).bfloat16().item())
```

完成判据：六个数全部打印；你的结论必须是：**bf16 归零比例为 0.0、fp16 有明显非零比例（我这边约 7e-4）**，**fp16 的相对误差比 bf16 小一个约 4 倍的量级**，`70000 → fp16 是 inf、bf16 仍在`；三个判断各配一个打印值写进 NOTES，并在 `week04/notes-training.md` 补一张 4 行表（`位宽构成 / 指数范围 / 尾数精度 / 是否需要 GradScaler / 4090 上选谁`）——其中「4090（Ampere）原生支持 bf16，指数范围与 fp32 相同所以不会溢出，

代价是尾数只有 7 位」这行是重点，最后一行写清「所以优先 bf16，不需要 loss scaling；fp16 必须配 GradScaler」。

> 注：fp16 的问题从来不是「精度不够」，而是**范围太窄**：最小正规数约 6e-5，小梯度直接下溢成 0；GradScaler 的做法是把 loss 乘一个大系数把梯度抬进可表示区间，`step` 前再除回来。**bf16 = fp32 的指数位 + 7 位尾数**，范围一样，所以不需要 scaler。这套推理你今天用 6 个打印值自己走一遍，比背 10 遍「bf16 更稳」有用。

任务 2：梯度累积的数值等价性（约 120 分钟）

`week04/day4/accum_check.py`（float64，`micro=1` 是整批一步，`micro=4` 是四步累积）：

```
import torch, torch.nn.functional as Fn
torch.manual_seed(5)
T, V, C = 8, 32, 16
xs = torch.randint(0, V, (4, T)); ys = torch.randint(0, V, (4, T))
def ce(emb, head, x, y):
    return Fn.cross_entropy(head(emb(x)).reshape(-1, V), y.reshape(-1))
def make(kind):
    torch.manual_seed(7)                                # 每次从同一初始参数出发
    emb = torch.nn.Embedding(V, C).double(); head = torch.nn.Linear(C, V).double()
    ps = [*emb.parameters(), *head.parameters()]
    opt = torch.optim.AdamW(ps, lr=1e-2) if kind == "adam" else torch.optim.SGD(ps, lr=1e-2)
    return emb, head, ps, opt
def chunks(micro):
    return [(xs[i::micro], ys[i::micro]) for i in range(micro)]   # 每个 micro-step 的 token 数必须相同
def run(kind, micro=4, divide=True, iters=20):
    emb, head, ps, opt = make(kind); log = []
    for _ in range(iters):
        opt.zero_grad(); tot = 0.0
        for x, y in chunks(micro):
            loss = ce(emb, head, x, y)
            (loss / micro if divide else loss).backward()
            tot += loss.item() / micro
        opt.step(); log.append(tot)
    return ps, log
def diff(a, b):
    return max((p - q).abs().max().item() for p, q in zip(a, b))
if __name__ == "__main__":
    print("AdamW 累积4步(/4) vs 整批一步:", diff(run("adam", 4, True)[0], run("adam", 1, True)[0]))
    print("SGD   不除 accum 的偏离      :", diff(run("sgd", 4, True)[0], run("sgd", 4, False)[0]))
    print("AdamW 不除 accum 的偏离      :", diff(run("adam", 4, True)[0], run("adam", 4, False)[0]))
    assert diff(run("adam", 4, True)[0], run("adam", 1, True)[0]) < 1e-9
    print("ACCUM EQUIV OK")
```

完成判据：打印 `ACCUM EQUIV OK`，第一行偏离 **<1e-9**（我这边 4.4e-16）；第二行（SGD 不除 accum）偏离在 **1e-2 量级**（我这边约 0.070）、第三行（AdamW 同样错误）在 **1e-6 量级**（我这边约 4.1e-6）；NOTES 必须回答：**为什么同一个 bug 在 AdamW 下几乎看不出来**（Adam 的更新量近似 $\hat{m}/\sqrt{\hat{v}}$，对梯度整体缩放不敏感——于是这个 bug 会变成「悄悄改了有效 lr 与 warmup 形状」而不是「明显跑偏」）。

> 注：等价的前提有两条，都很脆：① 各 micro-step 的 **token 数相同**（变长数据要用「按 token 数归一」而不是「按步数除」，Week 6 SFT 必撞）；② `dropout` 关掉或 `p=0`，否则两版抽到的掩码不同。我的脚本用 `float64` + `p=0` 就是为了

> 把等价性做成**逐位可比**，你加任何一条随机性都要能解释偏离。

任务 3：`clip_grad_norm_` 的 global norm 语义 + 日志（约 90 分钟）

在上面脚本之后接一段（**同一次反向的梯度**）：

```
emb, head, ps, opt = make("adam")
ce(emb, head, xs, ys).backward()
pre = torch.sqrt(sum((p.grad ** 2).sum() for p in ps)).item()
r1 = torch.nn.utils.clip_grad_norm_(ps, 1.0).item()
post1 = torch.sqrt(sum((p.grad ** 2).sum() for p in ps)).item()
r2 = torch.nn.utils.clip_grad_norm_(ps, 0.1).item()
post2 = torch.sqrt(sum((p.grad ** 2).sum() for p in ps)).item()
print(f"pre-clip={pre:.4f} 阈值1.0: 返回={r1:.4f} 之后={post1:.4f} 阈值0.1: 返回={r2:.4f} 之后={post2:.4f}")
assert abs(pre - r1) < 1e-6 and post2 <= 0.1 + 1e-6
print("CLIP SEMANTICS OK")
```

完成判据：打印 `CLIP SEMANTICS OK`；**返回值 == 裁剪前的 global norm**（两个数在你输出里必须相等，误差 <1e-6）；阈值 0.1 之后总范数被压到 ≤0.1；NOTES 写两句：① 「global norm 是所有参数拼成一个向量后的 L2 范数，所以它是**跨层联合裁剪**，单个大梯度会被整层拖着一起缩小」；② 「nanoGPT 用 `grad_clip=1.0`，且它把裁剪前的 norm 记进了日志——**没有这条日志，你看不到梯度尖峰，也就无法区分『lr 太大』和『数据里有毒样本』**」。

任务 4：把三件事接进 `train_v1.py` + 收工（约 60 分钟）

给 `train_v1.py` 加四个开关：`--dtype {fp32,bf16}`（bf16 用 `torch.autocast` 或直接把模型 `.to(torch.bfloat16)`，**CPU 不支持 bf16 autocast 时走后者，如实记录**）、`--accum N`（含 `/N` 与「有效 batch」打印）、`--clip 1.0`（并把裁剪前 norm 打进日志）。

完成判据：三个开关各自跑 30 步不报错，日志里同时有 `iter / loss / lr / grad_norm(pre-clip) / 有效 batch`；`--accum 4 --batch_size 8` 打印的有效 batch 是 32；NOTES 记一条今天最意外的现象（含打印值），并把 bf16 相关的验证排到明天卡上。

今日验收（6.5h）：bf16/fp16 六个数 + 表 4 行 + `ACCUM EQUIV OK`（三种偏离齐全）+ `CLIP SEMANTICS OK` + 三个开关可用。

Day 5 • 周五：显存解剖与 gradient checkpointing——今天上卡第一次

今日总览（6h）：上午本地算表，下午上卡（**计划 2.5h，含开卡与收尾**）：① 把「权重 / 梯度 / 优化器态 / 激活」四项算成一个数；② 在 4090 上实测 `memory_allocated` 与 `max_memory_reserved`，和手算对账；③ 同一模型开/关 gradient checkpointing，记一张显存/速度对照表；④ DDP/ZeRO 只到能讲清「各自切什么」。**今晚就把 Day 6 的启动脚本写好，明天开卡直接跑。**

任务 1：显存解剖表——先手算（约 80 分钟）

以 nanoGPT 默认 GPT-2 124M（`n_layer=12, n_head=12, n_embd=768, block_size=1024, vocab=50257`）为例，先验证参数量：`12·n_layer·C²`（每层 attn `4C²` + FFN `8C²`，Week 3 对拍过）+ `wte` 的 `V·C` + `wpe` 的 `block_size·C`，看是否落在 124M 附近。然后填这张表（顶层表格，单位 GB，按 1e9 字节）：

组成	每参数字节	124M 参数量下	备注
权重 fp32 / bf16	4 / 2	?	
梯度 fp32 / bf16	4 / 2	?	
AdamW 状态（m+v, fp32）	8	?	Day 3 算过
激活（≈ 与 `B·T·C·n_layer` 成正比）	—	?	今天只到「随 batch 与序列长线性/二次增长」的定性

完成判据：表填满，三个「?」处是**你算出来的数**（bf16 权重+梯度+fp32 优化器态合计应为 **12 B/参数 ≈ 1.49 GB**，纯 fp32 应为 **16 B/参数 ≈ 1.98 GB**）；NOTES 一句「为什么优化器态常是显存第一大头，而它只随参数量增长、不随 batch 增长」，一句「LoRA 为什么省显存（只训小矩阵 ⇒ 大部分参数没有梯度与优化器态）」——**这两句是 Week 6 的入场券**。

任务 2：上卡实测显存——`allocated` 与 `reserved` 不是一回事（约 80 分钟）

开卡（4090，**记下开卡时刻**），把 `week04/` 与模型脚本传上 `/root/autodl-tmp/week04`，跑：

```
source /etc/network_turbo
pip install numpy matplotlib tqdm -i https://pypi.tuna.tsinghua.edu.cn/simple
cd /root/autodl-tmp/week04 && python - <<'EOF'
import torch, time
from day3.adamw_check import Net                          # 换个真一点的模型也行,关键是能读参数总量
print(torch.__version__, torch.cuda.get_device_name(0))
EOF
```

真正要跑的是这段（放进 `week04/day5/mem_probe.py`）：构造一个参数量已知的模型（就用你 Day 2 的小 GPT，`print(sum(p.numel() for p in model.parameters()))`）→ 移到 cuda → 记录 `torch.cuda.memory_allocated()` → 一次前反向 + `opt.step()` → 再记录 → 用 `torch.cuda.max_memory_allocated()` 与 `max_memory_reserved()` 收尾。

完成判据：三个数打印（初始 allocated / 反向后 allocated / 峰值 allocated 与 reserved），**「反向后 − 初始」的增量与手算的「梯度 + AdamW 状态」相对差 <30%**（激活与缓存让它不可能精确相等，但量级必须对得上；差一个数量级就是你的表或统计口径错了）；`reserved ≥ allocated` 恒成立，NOTES 写一句为什么（分配器缓存 + 碎片，`empty_cache()` 才还给驱动）。

> 注：**`nvidia-smi` 显示的值 ≠ `torch.cuda.memory_allocated()`**，前者含 CUDA context 与分配器缓存，通常大几百 MB 到几 GB。Week 3 让你把这条记进 gaps，今天补上。另外：bf16 与 fp32 的对比要在**同一份脚本**里跑两次，别拿两次不同批次的数比。

任务 3：gradient checkpointing 显存/速度对照表（约 100 分钟）

`week04/day5/ckpt_speed.py`：8 层 `Linear(512,512)+GELU` 的堆叠，`batch=64`，`checkpoint(layer, x, use_reentrant=False)` 逐层包，跑 15 步取平均：

```
import time, torch, torch.nn as nn
from torch.utils.checkpoint import checkpoint
class Stack(nn.Module):
    def __init__(self, c=512, n=8):
        super().__init__()
        self.layers = nn.ModuleList([nn.Sequential(nn.Linear(c, c), nn.GELU()) for _ in range(n)])
    def forward(self, x, ckpt=False):
        for l in self.layers:
            x = checkpoint(l, x, use_reentrant=False) if ckpt else l(x)
        return x
def bench(ckpt, iters=15):
    torch.manual_seed(0); m = Stack().cuda()
    x = torch.randn(64, 512, device="cuda", requires_grad=True)
    opt = torch.optim.SGD(m.parameters(), lr=1e-4)
    for _ in range(3):
        opt.zero_grad(); m(x, ckpt).pow(2).mean().backward(); opt.step()
    torch.cuda.synchronize(); t0 = time.perf_counter(); peak = 0
    for _ in range(iters):
        opt.zero_grad(); m(x, ckpt).pow(2).mean().backward(); opt.step()
        torch.cuda.synchronize(); peak = max(peak, torch.cuda.max_memory_allocated())
    return (time.perf_counter() - t0) / iters, peak
if __name__ == "__main__":
    tf, pf = bench(False); tc, pc = bench(True)
    print(f"普通 {tf*1000:.1f} ms / 峰值 {pf/1e6:.0f} MB | checkpointing {tc*1000:.1f} ms / 峰值
{pc/1e6:.0f} MB")
    print(f"变慢 {(tc/tf-1)*100:.0f}%  显存降到 {pc/pf:.2f}")
    assert tc > tf, "checkpointing 反而更快？说明这块计算太便宜,把 c 或层数加大"
    print("CKPT SLOWER OK")
```

完成判据：打印 `CKPT SLOWER OK`，**耗时上升（预期量级 +30%~+80%）且峰值显存下降（预期 <0.6 倍）**——两个方向都必须对，具体数以你实测为准；把四格表（ms、峰值 MB、倍数）抄进 `week04/day5/mem_table.md`；NOTES 一句「省显存的原理：不保存中间激活，反向时重算 ⇒ 用算力换显存」+ 一句「为什么它和梯度累积是两种互补的手段」。

> 注：`use_reentrant=False` 是必须的——老的 reentrant 实现对 `non-tensor` 输入、对 `torch.no_grad` 内部调用都有坑，报错文案还很有误导性。CPU 上这个脚本也能跑（速度差仍在、显存差看不到），**抢不到 4090 就本地跑 CPU 版并把「显存两列待补卡」记进 gaps**，别为这张表硬开贵卡。

任务 4：DDP / ZeRO 概念表——只到「能讲清切什么」（约 50 分钟）

一张 5 行表（顶层表格）：`对象 | 数据并行 DDP | ZeRO-1 | ZeRO-2 | ZeRO-3`，行是「模型权重 / 梯度 / 优化器态 / 激活 / 数据分片」，每格写「复制 / 分片 / 不切」，并在表下各写一行「通信代价从哪来」（DDP 每步 all-reduce 梯度；ZeRO-3 前向反向都要 all-gather 参数，所以更吃带宽）。

完成判据：表 5 行 × 5 列填满，且**没有一格写「未知」**；NOTES 有一句能直接讲出口的对比：「DDP 省不了单卡显存（每份都是全量优化器态），ZeRO 的本质是把 Adam 的 12 B/参数摊到 N 卡上」——并能答「为什么 ZeRO-3 对网络带宽敏感」；再加一句「本阶段我用不到它们：1.5B LoRA 在 24G 上很舒适（总计划的结论），先记账，Week 15 之后再实操」。

任务 5：Day 6 启动脚本与收工（约 50 分钟）

关卡之前把明天要跑的东西全部就位：`week04/day6/run_card.sh`（数据准备 → 1000 步 → 存档 → 续跑 200 步 → pre/post 对照），本地 `bash -n` 过一遍语法。**今天累计卡时打印出来**（AutoDL 控制台或你记的开卡时刻）。

完成判据：`run_card.sh` 存在且不超过 20 行、每行有注释；NOTES 有今天的卡时数与累计数（本周累计 ≤6h 为合格，超了立刻写「Day 6 砍第二组」）；已关机（控制台显示「已关机」）。

今日验收（6h）：显存表三格有数且合计对上 12B/16B + `allocated/reserved` 两列实测 + checkpointing 两个方向都对 + DDP/ZeRO 表满格 + 卡已关。

Day 6 • 周六：输出日——`train_template.py` 定稿真跑、稳定性对照、闭卷自测、push

今日总览（6.5h）：今天把前六天的东西收成两个交付物：`week04/train_template.py`（以后每个项目都会抄的那一份）和`week04/踩坑记录.md`（Week 14 面试最值钱的材料）。卡上用约 2.5h：1000 步真跑 + 续训复现 + 一组 pre/post 稳定性对照。

任务 1：`train_template.py` 定稿并在 4090 上跑 1000 步（约 120 分钟）

把 Day 1-5 的东西合并成一个文件，要求功能齐全：`--dtype bf16` / `--accum 4` / `--clip 1.0` / `--resume` / `iter_num` **驱动的 warmup+cosine / 每 N 步 eval（`model.eval()` + `no_grad`）/ 显存与 `grad_norm` 日志 / `seed` 全套**。模型用你 `week03/block.py` 的小 GPT + 今天对齐好的 `scaled_init`、weight tying、`ln_f`、`dropout=0.1`。开卡前把命令写进 `run_card.sh`：

```
cd /root/autodl-tmp/week04 && python train_template.py --max_iters 1000 --dtype bf16 --accum 4 --clip 1.0
--out ck/w04
python train_template.py --resume ck/w04/step600.pt --max_iters 800 --dtype bf16 --accum 4 --clip 1.0
```

完成判据：1000 步跑通，loss 从 `ln(V)` 降到明显低于随机基线（**用 `exp(val_loss)/vocab_size < 1` 判断，和 Week 3 同一把尺**）；第 2 条命令从 `iter_num=600` 接着跑、**接上后前 20 步 loss 与第 1 条命令同区间的自然走势连续**（不出现跳回 4.x 或突然下跌）；日志里每步都有 `lr / grad_norm / 有效 batch / allocated MB`。抢不到卡 → 本地 CPU 跑 300 步（`--dtype fp32`），卡的部分记 gaps，**不许拿别人的曲线交差**。

> 注：「从 600 步续训」这条判据故意做得模糊——**loss 曲线跳变**是一个你需要自己解释现象的判断，我给不了阈值。真正常见的三种原因按排查顺序是：没存/没恢复 rng 与 optimizer state、lr 由「第几次调用 scheduler」而不是 `iter_num` 决定、resume 时数据窗口变了。

任务 2：Pre-LN vs Post-LN 的稳定性对照——把 Week 3 讲的变成你看到的（约 80 分钟）

同一份 `train_template.py`、同一份 config（`n_layer=6`），两组：① `mode=pre`、`lr=1.0`（荒谬地大）；② `mode=post`、同样 `lr=1.0`。各跑 300 步，每 10 步打印 loss 与 `grad_norm`，曲线各存一张 PNG。

完成判据：两组各一张图（或两条曲线同图），**至少有一组出现 `nan` / `inf` 或 loss 不降**（这是 Week 5 要练的「故意制造发散」的第一次排练；两组都稳定就把 lr 再乘 3 重跑，别改成「都能训」）；NOTES 写清**发散发生在第几步、`grad_norm` 在发散前一步是多少**，并给一句「Pre-LN 更稳的机制解释 + 我这次的实测是否支持它」——**如果实测不支持我的说法，写下来，那比顺着说更有价值**。

任务 3：白板默写训练循环 + 踩坑记录（约 60 分钟）

纸上默写完整训练循环（从 `set_seed` 到 `save_ckpt`），标出：`zero_grad` 的位置、`/accum` 的位置、`clip` 在 `backward` 之后 `step` 之前、`lr` 从哪来、eval 的两个开关。然后写 `week04/踩坑记录.md`：**每条「现象 → 定位 → 修复」，≥6 条，全部来自本周你自己的终端**（含 Day 1 的 Windows 多进程、Day 3 的对拍不等价）。

完成判据：默写稿与 `train_template.py` 逐项对照，**顺序错 0 处**（`clip` 放在 `step` 之后就是顺序错，`clip` 之后又 `backward` 也是），错一处就地重默；踩坑记录 ≥6 条且每条有报错原文或数字，**一条「别人告诉我的」都不算**。

任务 4：闭卷自测 4 题与订正（约 70 分钟，计时 55 分钟做题）

1. 写一个完整的训练循环骨架（含累积、裁剪、eval、存档五样），标出每处「顺序」上的约束并说明违反的后果。
2. `micro-batch=4`、每步 8 token、`accum=4`：写出 loss 该怎么归一；再说 dropout 开着时累积与整批是否等价、为什么。
3. bf16 为什么不需要 GradScaler 而 fp16 需要？fp16 下小梯度会发生什么，GradScaler 具体在做什么？
4. 续训后 loss 比连续训练高 0.03，列一个**按成本排序**的排查清单（先查什么后查什么），并说明你打算用什么实验证实/排除每一条。

完成判据：四题都写、计时 ≤55 分钟；第 1、4 题是硬指标（第 1 题写不出 → 今晚重做 Day 2 任务 5；第 4 题若只写了 ≤2 条或没有「用什么实验验证」→ 重做 Day 2 任务 3 的两组对照）。

任务 5：repo 收口与 gaps 新洞映射（约 60 分钟）

1. `week04/` 整理成：`train_template.py`、`data_lab.py`、`resume_test.py`、`adamw_check.py`、`sched_check.py`、`accum_check.py`、`dtype_lab.py`、`ckpt_speed.py`、`mem_table.md`、`踩坑记录.md`、`README.md`（四段：是什么 / 怎么跑 / 关键结论 / 诚信声明）。拷进 `llm-posttraining-lab/notes/week04-training/` 并 push。
2. 更新 `week04/week04-gaps.md`（≥6 条），新洞必须落进下表：

本周暴露的洞	归到	为什么那天补上
只在玩具模型上验证过等价性，真实 124M/更大模型的显存与吞吐没测过	**Week 5 Day 2-3（M1）**	M1 第一次真训练就要看 `it/s` 和峰值显存来定 config
变长样本的累积归一（按 token 数除）只写了结论没做实验	**Week 6 Day 2（SFT loss masking）**	SFT 每条样本长度不同、还要 mask 掉 prompt，`/accum` 那套必须重来
tokenizer（BPE）与 `block_size` 的真实影响	**Week 5 Day 1**	M1 数据准备第一步就是选 char/bpe
bf16 下 `grad_norm` 与 fp32 的偏差只有小模型证据	**Week 5 Day 4（发散演练）**	故意大 lr 发散时会在同一份日志里对拍
采样生成（temperature/top-k）与 cache 仍未碰	**Week 5 Day 5**	M1 交付要采样样例
DDP/ZeRO 只有概念、`torchrun` 一次没跑	**Week 15 之后（分布式补强档）**	若已转社招再按总计划风险 4 加做实操

完成判据：GitHub 链接可访问、README 渲染正常、**在 clone 出来的目录里** `python adamw_check.py` **打印** `ADAMW CROSSCHECK OK`、`python accum_check.py` **打印** `ACCUM EQUIV OK`（在自己机器上跑通才算交付）；gaps ≥6 条且每条有「Week 几 + 哪天」。

今日验收（6.5h）：模板 1000 步 + 续训衔接 + 至少一组发散 + 默写 0 顺序错 + 自测 4 题有判分 + push 成功。

Day 7 • 周日：缓冲补漏 + 闭卷复测 + 总验收 + Week 5（M1）预习

今日总览（4h）：不排新内容。补漏最多 2 个洞、闭卷复测 3 题、做总验收、预习下周。**下周是 M1 硬里程碑，交付时间不可顺延**，今天必须把卡订到、把数据准备好。

任务 1：补漏（约 70 分钟）

打开 `week04-gaps.md` 与 `踩坑记录.md`，**只补 2 个洞**，优先：① Day 6 任务 1 的续训衔接如果没做到「走势连续」，今天就重做一遍并找出原因；② 显存表里那格「激活」如果只有定性描述，补一次实测（同一模型 `B=1` vs `B=8`，看峰值差与什么成正比）。

完成判据：两条在 gaps 里变 `[x]`，各附「怎么补的 + 复跑打印了什么」；若第 ② 条要在卡上做，写明「已并入 Week 5 Day 2」而不是留着不处理。

任务 2：闭卷复测 3 题与订正（约 60 分钟）

合上一切重做：① 默写训练循环（累积 + 裁剪 + eval 双开关 + 存档五样）；② 三句话分别解释 bf16 vs fp16、`/accum` 为什么必须、`clip_grad_norm_` 裁的是什么；③ 给一个「续训后 loss 不对」的场景，写出 3 条

排查假设 + 各自的验证实验。计时 35 分钟，余下订正。

完成判据：① 顺序 0 错；② 每句 ≤2 行且各带一个本周实测过的数；③ 每条假设都配一个实验（写「再看看」不算）；订正写在原卷上，结果进 `review.md`。若 ① 又错，**把 Week 5 Day 1 的前 60 分钟改成重做本周 Day 2 任务 5**，并在 `week04-gaps.md` 顶部写明。

任务 3：总验收（约 30 分钟）

#	验收项	判据
1	数据管线	`GETBATCH OK` + `DATALOADER OK`，`len(dl)` 与手算相等，且有一个带数字的选型结论
2	训练循环	200 步 loss ≤ 起始 80%；`train/eval` 三行 `True/False/False` 打印正确
3	checkpoint 续训	`RESUME OK`（恢复 rng 后 diff 为 0 或 1e-15 量级、不恢复 >1e-4）；模板从 `step600` 续跑走势连续
4	手写 AdamW	`ADAMW CROSSCHECK OK`，3 个 lr 偏离都 <1e-6；AdamW 状态 = 8 B/参数能口头推
5	lr 调度	`SCHED CROSSCHECK OK`（diff <1e-12）+ `lr_sched.png` 端点值与手算一致 + 续训 lr 序列逐位相同
6	精度 / 累积 / 裁剪	bf16 与 fp16 六个数齐全；`ACCUM EQUIV OK`（三种偏离）；`CLIP SEMANTICS OK`（返回值 = 裁剪前 norm）
7	显存与真跑	显存解剖表合计对上 12 B/16 B；`allocated` vs `reserved` 有实测；`CKPT SLOWER OK`（变慢且显存下降）；`exp(val_loss)/vocab < 1`；repo 在 clone 目录里能复跑对拍

≥6 项通过 = 本周合格；第 3/4/6 项不许缺——它们是 Week 5 M1 训练和 Week 6 SFT 的直接前置。

完成判据：验收表 7 行都有状态与一行原因（写「没做」必须带归派周），抄进 `review.md` 末尾。

任务 4：Week 5（M1）预习与收工（约 80 分钟）

1. 读总计划里 M1 的验收标准，把「交付清单」抄进 `week05/day1/todo.md`：数据准备（TinyStories 或字符级降级）、config 逐项与显存关系、发散演练、eval loss/PPL、采样对照小表、博客《我从零训了一个 GPT》。
2. 翻 nanoGPT `train.py` 与 `sample.py` 各 15 分钟，回答三件事：`eval_iter / eval_interval` 怎么控制验证集大小、`init_from = resume` 时它恢复了哪几样、`sample.py` 里 temperature 与 top-k 作用在哪两行。
3. **订好下周的卡**（Week 5 预算 ≤12h ≈ ¥25-30，P1 累计到本周结束时不能超过 30h）；把 `week04/train_template.py` 复制一份成 `llm-posttraining-lab/m1-nanogpt/` 的起点，README 写一行「M1 从这里长出来」。
4. 收工：`git status` 干净、gaps 每条有归属、素材进 `blogs/drafts/week04-materials/`（**本周不发**）。

完成判据：`todo.md` 六项齐全；nanoGPT 三问各 ≥2 行且带 `文件:行号`；下周卡已订且 NOTES 写了预算数；`blogs/drafts/week04-materials/` ≥3 个文件。

今日验收（4h）：验收表 ≥6 项 + 复测 ① 顺序 0 错 + 预习三问有行号 + 下周卡已订。

三、常见问题与纪律

- `OMP: Error #15` / `libiomp5md.dll already initialized`：conda base 与 torch 自带 OpenMP 打架。正解是回 `E:/study/posttrain/.venv`；临时绕过 `set KMP_DUPLICATE_LIB_OK=TRUE`。**在 venv 里还能复现的话，通常是同时装了 conda 版 `numpy/mkl` 和 pip 版 torch**，把 conda 那份卸掉。
- **对拍不等价（AdamW / 累积 / 调度器）的排查顺序**：① 两边 dtype 是否都 float64 ② 步数计数起点（`t` 从 0 还是 1，偏差修正是 `1-b^t`）③ 权重衰减是否解耦（`p *= 1-lr*wd` vs 塞进 grad）④ `eps` 在根号内还是外 ⑤ 初始参数与数据是否同一份（两个 `build()` 之间必须重放同一种子）⑥ `opt.step()` 顺序与 `zero_grad` 位置 ⑦ dropout 是否开着（等价性测试必须 `p=0`）。**先把偏离打印出来再 assert，只贴 `AssertionError` 等于没查。**
- **显存看哪个数**：`torch.cuda.memory_allocated()` 是活跃张量，`max_memory_allocated()` 是峰值，`reserved` 是分配器向驱动要的量，`nvidia-smi` 还含 CUDA context。**做对比必须同一进程、同一口径**；要观察分配器行为，加 `torch.cuda.empty_cache()` 前后各打印一次。
- **卡时超了**：本周上限 8h。Day 6 那 1000 步如果超过 40 分钟没结束，八成是 `block_size` / `batch_size` 或 `accum` 写错（这个模型很小），直接 Ctrl+C、**先关机**、把现场记进 gaps，别烧钱等结果；4090 无货就换任意有货型号（**别刷页面等卡**），连不上卡就把 Day 5/Day 6 的 GPU 列留空跑 CPU 版并写明「待补卡测」。
- **某天没完成**：不熬夜追。按「天-任务号」写进 `week04-gaps.md`，Day 7 是留给它们的。连续两天崩就砍 Day 1 任务 5 的计时对照与 Day 5 任务 4 的 DDP 表（**推到 Week 15**），**永远优先保 Day 2 的续训一致性、Day 3 的 AdamW 对拍、Day 4 的累积等价性**——这三项是 Week 5 出事时唯一能救你的东西。
- **AI 工具边界**：允许让 AI 解释报错、讲概念、review 写好的文件；**禁止**让它生成 `train_template.py` / `adamw_check.py` / `resume_test.py`——这三个文件是本周的肌肉记忆，也是 Week 13 手撕题的原题。允许在对拍全绿之后让 AI 逐行讲 `torch.optim.AdamW` 的实现细节，但要先把 NOTES 里那三处坑自己写完。
- **诚信与合规**：README 写清哪些跟敲 Karpathy/nanoGPT、哪些自己写；本周只用公开莎士比亚数据，**Week 7 起涉及明日记忆真实数据，脱敏规则开始生效**（PII 正则 + 人工抽检，用户可识别信息一律不进训练集）。